Ginhawa station: remaining build steps

These pick up where the battery pack, power board, fuse + INA219 and solar charger steps left off. Same format: what you need, numbered steps, a **Check** after each part, and what to do if it fails.

Build order

#	Stage	Status
1	Battery pack	done / sent
2	Power board (+ wire on row 17)	+ wire done
3	Fuse + INA219	sent
4	Solar panel + CN3791	sent
A	**Finish the power board (− wire on row 02)**	below
B	**3.3 V converter (TPS63020)**	below
C	**5 V converter (MT3608)**	below
D	**ESP32 board**	below
E	**Arduino IDE + first test (Blink)**	below
F	**SEN55 + INA219 wiring and test**	below
G	**4G modem (A7670SA) wiring and test**	below
H	**Start the Ginhawa app on the laptop**	below
I	**Upload the Ginhawa program**	below
J	**24-hour table test**	below
K	**Prepare the box**	below
L	**Put everything in the box**	below
M	**Mount it outside and check the app**	below

Your multimeter (ANENG M118A): quick reminder

You want	Press SELECT until the screen shows	Read it as
Voltage	**V** with the straight-line (DC) symbol	the number is the voltage
Beep (is it connected?)	the **•)))** speaker symbol	**beep = connected**, silence = not connected
Resistance	**Ω**	the number is the resistance

Black probe in **COM**, red in **VΩ** (right socket). Never the 10A socket. **Beep and Ω tests only with the power OFF** (fuse out).

Heat-shrink and wire tags

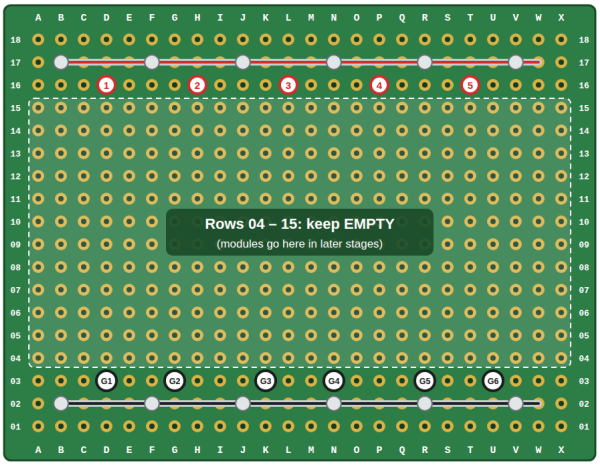
Where	Tube
22 AWG joints	**2 mm**
18 AWG joints	**2 mm** if it slides over, else **3 mm**
Twisted joints	**4 mm**

Wire	Colour	Tag at both ends
3.3 V (TPS63020 output)	red	**yellow**
5 V (MT3608 output)	red	**blue**
SDA	black	**yellow**
SCL	black	**blue**
Modem TXD / RXD / PWRKEY	black	masking-tape flag "TX" / "RX" / "PWR"
All grounds	black	no tag

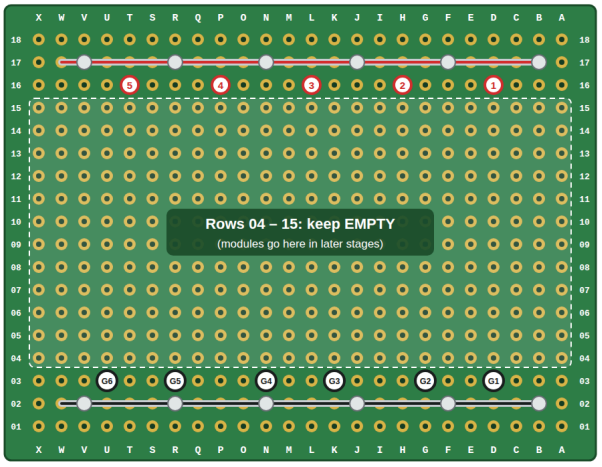
Power board taps (where each wire goes)

Ginhawa power board (5 × 7 cm): where the bus wires go

TOP SIDE (printed letters readable, A on the left, 18 at the top)



BOTTOM SIDE (flipped left-to-right: letters are mirrored, X on the left)



VBAT bus: bare red-marked wire on ROW 17, from B17 to W17

GND bus: bare black-marked wire on ROW 02, from B02 to W02

Solder the bus wire here: columns B, F, J, N, R, V (every 4th hole)

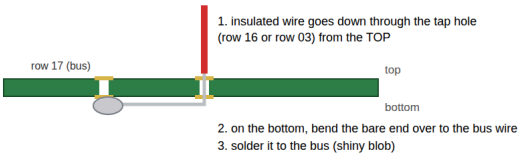
VBAT taps 1–5 (row 16: D16 H16 L16 P16 T16) – used in later stages

GND taps G1–G6 (row 03: D03 G03 K03 N03 R03 U03) – used in later stages

What plugs into each tap later

1	D16	INA219 VIN−	Stage 13	G1	D03	BMS P−	Stage 13
2	H16	CN3791 BAT +	Stage 14	G2	G03	CN3791 BAT −	Stage 14
3	L16	TPS63020 VIN	Stage 15	G3	K03	TPS63020 GND	Stage 15
4	P16	MT3608 VIN+	Stage 16	G4	N03	MT3608 VIN−	Stage 16
5	T16	A7670SA VBAT	Stage 21	G5	R03	A7670SA GND	Stage 21
				G6	U03	ESP32 board GND	Stage 17

How a tap wire joins the bus (later stages)



CHECK with the multimeter on BEEP (no battery connected):

```
B17 ↔ W17 → BEEP (VBAT bus is one wire)
B02 ↔ W02 → BEEP (GND bus is one wire)
row 17 ↔ row 02 → NO beep (no short between + and −)
row 17 ↔ any middle hole (e.g. L10) → NO beep
```

Power board taps: + wire row 17 (taps 1–5 on row 16), − wire row 02 (taps G1–G6 on row 03).

Every tap works the same way: the wire goes in from the **front** through its hole, and on the **back** you bend the bare end onto the bus wire and solder it.

Stage A – Finish the power board (− wire)

Time: 15 minutes.

What you need

- Power board (+ wire already on row 17)
- 7 cm of **black** 18 AWG
- Iron, solder, flux, stripper, pliers, masking tape, multimeter

Steps

1. Strip **all** the plastic off the 7 cm black wire, straighten it with two pliers, and tin it.

2. Turn the board to the **back** (letters read backwards). Find **row 02**: the **second row from the bottom**.

3. Lay the wire **on top of the row 02 holes**, along their centres, from the **2nd hole from one edge** to the **2nd hole from the other edge**. Tape it at both ends.

4. Solder it to the rings at about **every 4th hole** (more is fine). Keep every joint **on row 02 only**.

5. Remove the tape; press the wire with a fingernail: it must not lift.

✅ Check (fuse out, meter on •))))

Touch the probes to	Should
Both ends of the − wire	**BEEP**
− wire and each hole in **row 01** and **row 03**	**silence**
+ wire (row 17) and **− wire (row 02)**	**silence**

If it beeps where it shouldn't: a bridge. Clean the tip, add flux, drag the iron through the gap, or use the desoldering pump. Test again.

Stage B – 3.3 V converter (TPS63020)

What it does: turns the battery's 3.0 – 4.2 V into a steady **3.3 V** for the ESP32.

Time: 20 minutes.

What you need

- **TPS63020** board (the 3.3 V one)
- **22 AWG** red and black, about 30 cm each
- 2 mm heat-shrink, **yellow** tag pieces
- Foam tape, iron, solder, flux, stripper, multimeter

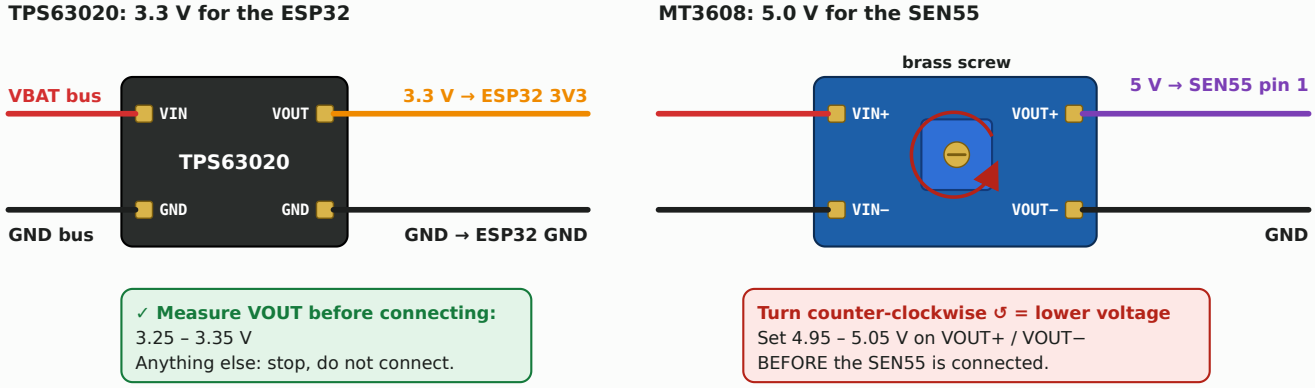


Figure 1. The two voltage converters. Both take power from the VBAT bus. Measure their outputs before anything is connected to them.

Steps

1. Fuse out.

2. Find the 4 pads on the TPS63020 (the names may be on the back): **VIN** and **GND** on the input side, **VOUT** and **GND** on the output side. Take a photo for reference.

3. Stick the TPS63020 on the **front** of the power board, in the empty middle area (rows 04 – 15), with a small piece of **foam tape**.

4. Input wires (10 cm each): - red 22 AWG from **VIN → tap 3 = hole L16** (+ wire); - black 22 AWG from input **GND → tap G3 = hole K03** (− wire).

5. Output wires (20 cm each): - red 22 AWG on **VOUT**, with a **yellow tag** at both ends: this is the **3.3 V wire**; - black 22 AWG on output **GND**.

6. Tape the free ends of the two output wires **separately** so they can't touch.

7. Cells in, **fuse in**. Meter on **V**: measure the yellow-tagged wire (red probe) against its black wire.

✅ **Check**

Measure	Expected
3.3 V wire to its black wire	**3.25 – 3.35 V**

8. Fuse out.

If it fails

What you see	Fix
About **5 V**	It's the 5 V version. **Don't use it for the ESP32.** Contact the seller.
0 V	VIN and GND swapped, or a tap not soldered. Fuse out, check with the beep setting.
Power board reads 0 V too	The BMS may still be asleep: connect the solar charger in sun first.

Stage C – 5 V converter (MT3608)

What it does: raises the battery voltage to **5 V** for the SEN55 sensor. **It arrives set much too high**, so you must adjust it before the sensor is ever connected.

Time: 20 minutes.

What you need

- **MT3608** board
- **22 AWG** red and black, about 30 cm each
- 2 mm heat-shrink, **blue** tag pieces
- **Small flat screwdriver**, nail polish or hot glue
- Foam tape, iron, solder, multimeter

Steps

1. Fuse out. Stick the MT3608 on the front of the power board next to the TPS63020 (foam tape).

2. Input wires (10 cm each): - red from **VIN+ → tap 4 = hole P16**; - black from **VIN− → tap G4 = hole N03**.

3. Output wires (20 cm each): - red on **VOUT+**, with a **blue tag** at both ends: this is the **5 V wire**; - black on **VOUT−**.

Tape the free ends separately.

4. Cells in, **fuse in**. Meter on **V**, probes on VOUT+ and VOUT−. It probably reads **6 – 28 V**.

5. Turn the **brass screw** on the blue box **counter-clockwise**, one full turn at a time. **It can take 10 – 25 turns before anything changes.** That's normal.

6. Stop at **5.0 V** (between 4.95 and 5.05). Too low? Turn clockwise a little.

7. Fuse out, fuse in, measure again: still 5.0 V.

8. Put a drop of **nail polish or hot glue** on the brass screw so vibration can't turn it.

9. Fuse out.

✅ **Check**

Measure	Expected
VOUT+ to VOUT−	**4.95 – 5.05 V**

⛔ **Never skip this.** More than 5.5 V destroys the SEN55 instantly.

If it fails

What you see	Fix
Doesn't change after 30 turns	Turn the other way 30 turns; some boards are reversed.
Stuck at about 3.7 V	Input wires swapped or a faulty board.

Stage D – ESP32 board

What it is: the ESP32 plugs into sockets on the **7 × 9 cm** board, so it can be removed to program it. The board also has a **3.3 V rail**, a **GND rail** and two small resistors.

Time: 1 – 2 hours.

What you need

- **7 × 9 cm** perfboard (not the practice one if you damaged it)
- **ESP32** (only to set the spacing, then removed)
- **Female header strip**, 2 × **4.7 kΩ** resistors
- 18 AWG black (25 cm), 22 AWG red and black, bare wire for rails
- Iron, solder, flux, cutter, pliers, multimeter

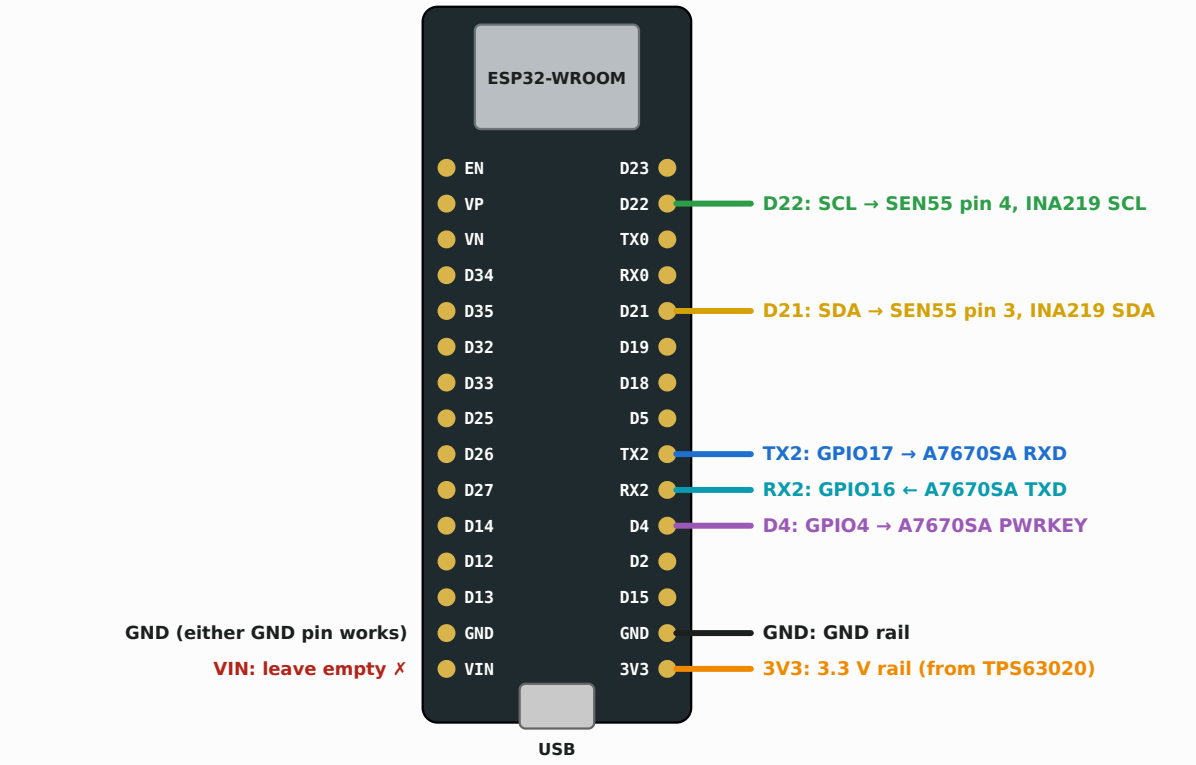


Figure 2. The ESP32 pins this project uses, all on one side of a 30-pin DevKit V1. **TX2 = GPIO17** and **RX2 = GPIO16** on the board's labels.

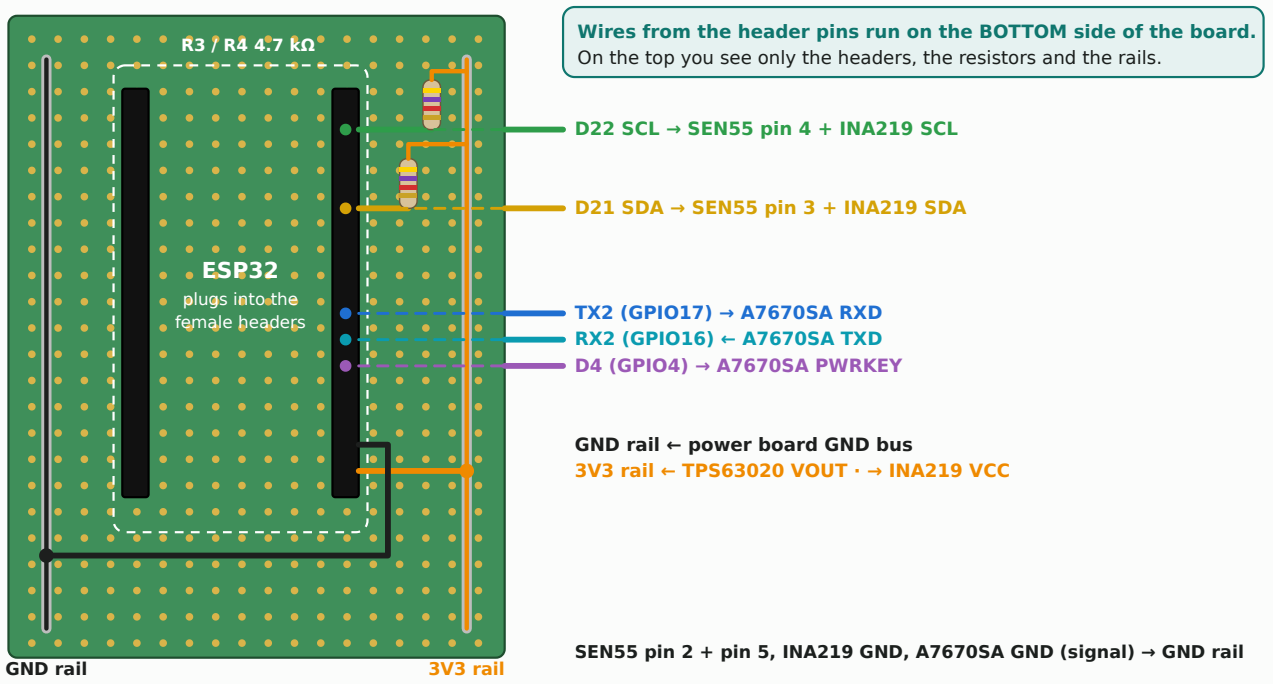


Figure 3. The ESP32 board (7 × 9 cm). Two rails (GND and 3V3), two pull-up resistors, and solder points for the sensor and modem wires.

Part 1 – Headers (40 min)

1. Count the pins on **one side** of your ESP32 (usually **15**).

2. Cut **two** pieces of female header that long: **pull the metal pin out** where you want to cut, then cut through the empty slot and file it smooth.

3. Push both headers onto the ESP32's pins. Set the ESP32 with its headers onto the board, **centred**, USB end at one short edge. This gives the exact spacing.

4. Hold it and turn the board over. **Solder one pin at each end of each header.** Turn it back: check the headers sit **flat and straight**. Then solder **all the other pins**. - With your 1 mm solder: **flux, a tiny dab per pin**, every other pin first.

5. Pull the ESP32 out and put it back in its bag.

> ✅ **Check:** meter on •))), touch each header pin to its neighbour: **silence everywhere** (no bridges).

Part 2 – Rails (20 min)

6. Make two bare, tinned wires (like the power board buses), as long as the board's long edge.

7. On the **back**, solder one along the **left long edge**: the **GND rail**. Solder the other along the **right long edge**: the **3V3 rail**. Mark them on the front with marker ("GND" black, "3V3" red).

8. On the back, with short pieces of wire: - the header pin under the ESP32's **3V3** pin → **3V3 rail**; - the header pin under the ESP32's **GND** pin → **GND rail**.

(Figure above shows where 3V3 and GND are: bottom of the right side, next to each other.)

Part 3 – Pull-up resistors (15 min)

9. Find the header pins under **D21** and **D22**.

10. Solder one **4.7 kΩ resistor** from the **D21** pin to the **3V3 rail**, and another from the **D22** pin to the **3V3 rail**. Resistors have no direction.

Part 4 – Power in (15 min)

11. The **yellow-tagged 3.3 V wire** from the TPS63020 → **3V3 rail**. Its black output wire → **GND rail**.

12. 18 AWG black, 25 cm: one end to the **GND rail**, the other to **tap G6 = hole U03** on the power board.

> ✅ **Check (ESP32 NOT plugged in)**
>
> | Test | Meter | Expected |
> |---|---|---|
> | 3V3 rail to GND rail, **fuse in** | V | **3.25 – 3.35 V** |
> | D21 pin to 3V3 rail, **fuse out** | Ω | about **4.7 kΩ** |
> | VIN pin to anything, **fuse out** | •))) | **silence**: VIN stays empty |

13. Fuse out.

If it fails

What you see	Fix
Headers crooked	Reheat one pin at a time while pressing straight, or remove with the pump and redo.
3V3 rail 0 V	TPS63020 output wire or the U03 ground link not soldered. Check with •))).

Stage E – Arduino IDE + first test (Blink)

What it does: installs the free program that uploads code to the ESP32, and proves the laptop, cable and ESP32 work. Do this on the laptop; it doesn't need the rest of the station.

Time: 45 minutes.

What you need

- Laptop with internet
- **ESP32** (on its own, not on the board)
- **USB data cable** (one that can transfer phone files, not charge-only)

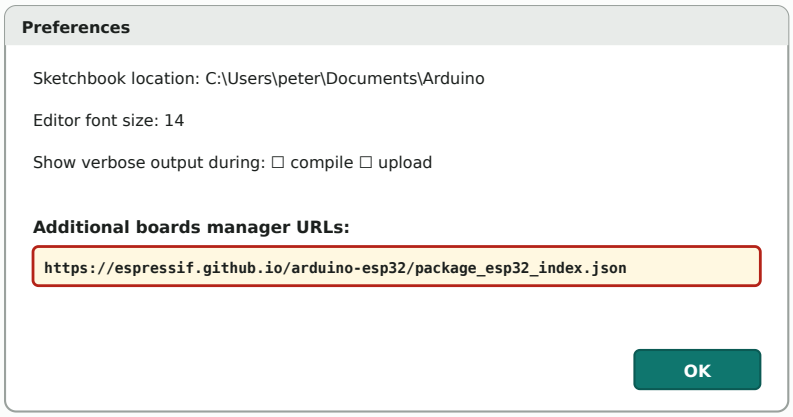


Figure 4. Adding the ESP32 boards address in Preferences. You do this once per computer.

Steps

1. Go to **arduino.cc/en/software**. Download **Arduino IDE 2** for Windows (64-bit installer). Install with all defaults; allow drivers if asked.

2. Open it. **File → Preferences**. In **"Additional boards manager URLs"** paste: `https://espressif.github.io/arduino-esp32/package_esp32_index.json` Click **OK**.

3. Click the **board icon** (2nd on the left bar). Search **esp32**. Install **"esp32 by Espressif Systems"**. Wait until it says INSTALLED (large download).

4. Click the **books icon** (3rd on the left). Install these, and click **"Install all"** when asked about dependencies: - **Sensirion I2C SEN5X** - **Adafruit INA219** - **ArduinoJson** (by Benoit Blanchon, version 7)

5. Plug the ESP32 into the laptop. Open **Device Manager** (right-click Start). Under **Ports (COM & LPT)**, a new **COM** number appears (e.g. COM3).

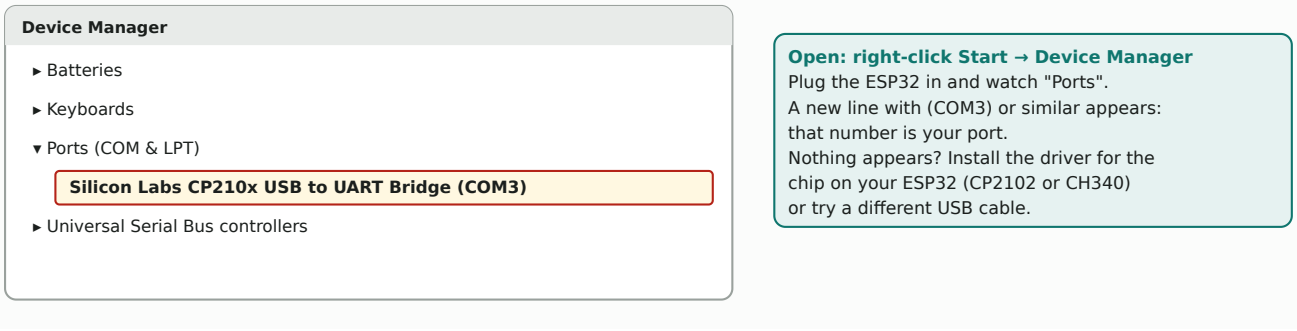


Figure 5. Finding the ESP32's COM port on Windows. The name says CP210x or CH340 depending on the USB chip on your board.

6. No COM port? Try another cable. Then look at the small chip next to the ESP32's USB socket: install the **CP210x** (Silicon Labs) or **CH340** (WCH) driver it names, restart, and plug in again.

7. File → New Sketch. Delete everything, paste the Blink code below.

```
#define LED_BUILTIN 2
void setup() { pinMode(LED_BUILTIN, OUTPUT); }
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); delay(1000);
  digitalWrite(LED_BUILTIN, LOW);  delay(1000);
}
```

8. Top bar → **Select other board and port** → choose **ESP32 Dev Module** and your **COM** port.

9. Click **Upload** (→). If it stays at **"Connecting...."**, **hold the BOOT button** on the ESP32 until percentages appear.

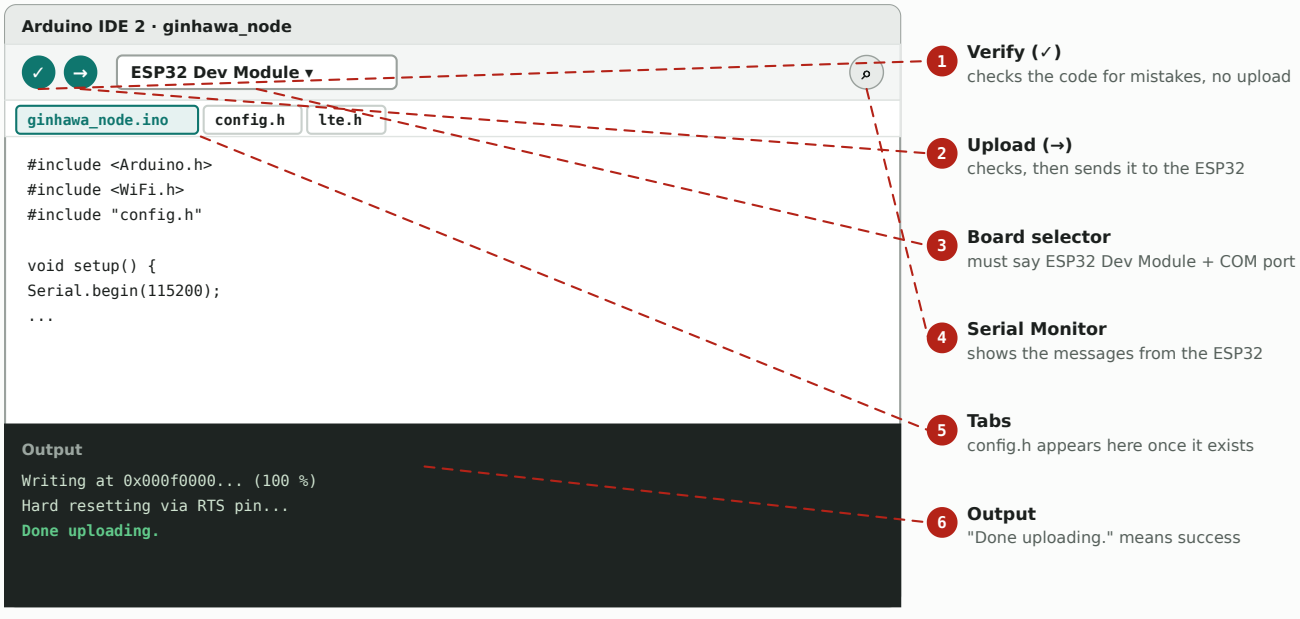


Figure 6. The Arduino IDE 2 window and the six things you will use. (Simplified drawing; colours on your screen may differ.)

✅ **Check:** "Done uploading" and the ESP32's **blue LED blinks** once per second.

If it fails

What you see	Fix
Port greyed out	Another program (e.g. Serial Monitor) is using it; close it.
"Failed to connect"	Hold BOOT during "Connecting"; or Tools → Upload Speed → 115200.
Uploads but no LED	Some boards have no LED on pin 2. "Done uploading" means it worked.

Stage F – SEN55 + INA219 wiring and test

What it does: connects the air sensor and the battery meter to the ESP32 using **two shared wires**: **SDA** (D21) and **SCL** (D22).

Time: 1 – 2 hours.

What you need

- **SEN55**, one **JST GH 6-pin cable**
- INA219 (already on the power path)
- 22 AWG black, yellow and blue tag pieces, 2 mm heat-shrink
- Iron, solder, flux, stripper, multimeter, laptop

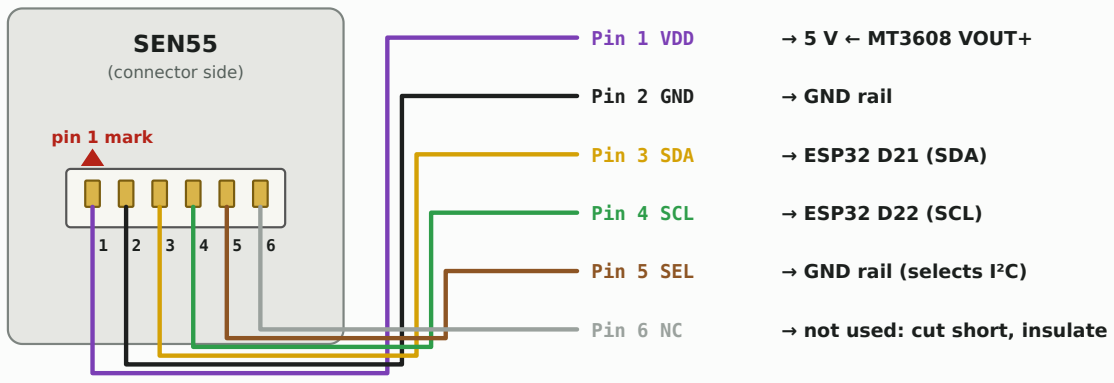


Figure 7. SEN55 pin-out and where each wire goes. Find **pin 1** from the mark printed next to the sensor's connector, then count. Go by pin number, never by wire colour.

Part 1 – Find pin 1 (20 min)

1. Cut **one** double-ended cable exactly **in the middle**. Keep the other half as a spare.

2. Strip **4 mm** of all 6 wires on one half and tin them.

3. Plug it **gently** into the SEN55 (line it up; never force it).

4. Find **pin 1**: a small triangle or "1" next to the SEN55's socket.

5. Meter on •))). One probe on a bare wire end; with the other, touch the metal contact inside the plug at the pin 1 position (a needle helps). Find which wire is pin 1, then 2 – 6.

6. Write it down: pin 1 = (colour), pin 2 = (colour), … **Wire colours on these cables aren't standard: always go by pin number.**

Part 2 – Solder the SEN55 (25 min)

7. Fuse out.

SEN55 pin	Connect to
1 VDD	**blue-tagged 5 V wire** (MT3608 VOUT+)
2 GND	**GND rail** (ESP32 board)
3 SDA	header pin under **D21** (yellow tag)
4 SCL	header pin under **D22** (blue tag)
5 SEL	**GND rail**
6 NC	nothing: cut short, cover with heat-shrink

8. Also solder the MT3608's black **VOUT−** wire to the **GND rail**.

9. Cover every joint with 2 mm heat-shrink.

Part 3 – Solder the INA219 signals (15 min)

10. Four 22 AWG wires from the INA219's pins:

INA219 pin	Connect to
VCC	**3V3 rail**
GND	**GND rail**
SDA	header pin under **D21** (yellow tag)
SCL	header pin under **D22** (blue tag)

Part 4 – Test (20 min)

11. Plug the **ESP32 into its headers**: every pin in a socket, none outside, USB end toward the board edge.

12. Cells in, **fuse in**, then plug the USB cable into the laptop.

13. Arduino IDE: **File → New Sketch**, paste **Test 1** below, **Upload**.

Test 1: I²C scanner

```
// Ginhawa test 1: finds devices on the I2C bus (SDA = D21, SCL = D22)
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  delay(1000);
  Wire.begin();  // SDA 21, SCL 22
}

void loop() {
  Serial.println("Scanning...");
  int found = 0;
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.printf("  found 0x%02X", addr);
      if (addr == 0x40) Serial.print("  <- INA219 (battery meter)");
      if (addr == 0x69) Serial.print("  <- SEN55 (air sensor)");
      Serial.println();
      found++;
    }
  }
  if (found == 0) Serial.println("  nothing found: check SDA/SCL, power and GND");
  Serial.println();
  delay(3000);
}
```

14. Open the **Serial Monitor** (magnifier, top right). Set it to **115200 baud**.

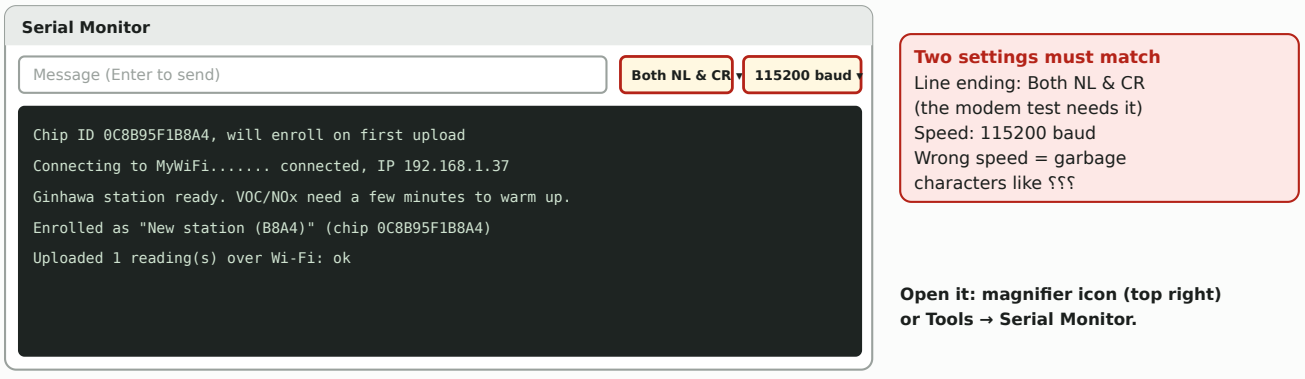


Figure 8. The Serial Monitor shows what the ESP32 is doing. The text is an example of a healthy first start.

✅ **Check:** the Serial Monitor shows, every 3 seconds: `found 0x40 <- INA219 (battery meter)` `found 0x69 <- SEN55 (air sensor)`

If it fails

What you see	Fix
Nothing found	Swap the D21/D22 wires; check the 3V3 rail; check the U03 ground link.
Only 0x40	SEN55 not powered or wrong pin order: measure 5 V between pin 1 and pin 2 wires.
Only 0x69	INA219 VCC or its SDA/SCL wires.
Strange characters	Serial Monitor not at 115200.

Stage G – 4G modem (A7670SA) wiring and test

What it does: sends readings over 4G when there's no Wi-Fi, and its GPS finds the station's location.

Time: 1 – 2 hours.

What you need

- **A7670SA** board, **LTE antenna**, **GPS antenna**
- **Nano-SIM** (registered, with data load)
- 2 × **1000 µF** capacitors
- 18 AWG red and black (15 cm each), 22 AWG black, masking tape
- Iron, solder, multimeter, laptop

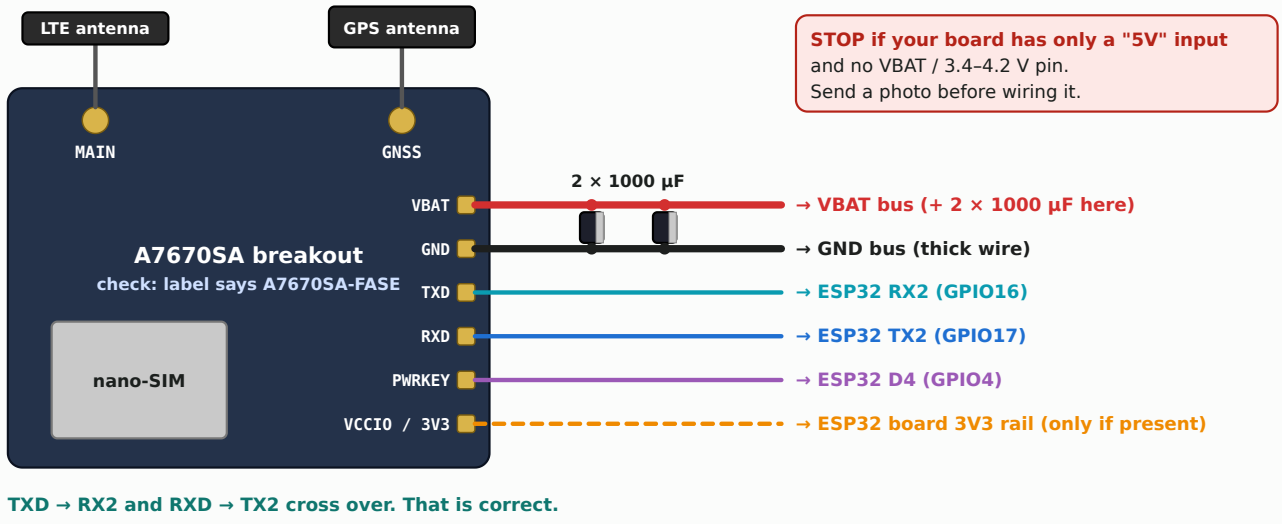


Figure 9. A7670SA connections. Pin names differ between breakout makers: PWRKEY may be printed as **PWR**, **PEN** or **K**; VBAT may be **VCC 3.4–4.2 V** or **BAT**.

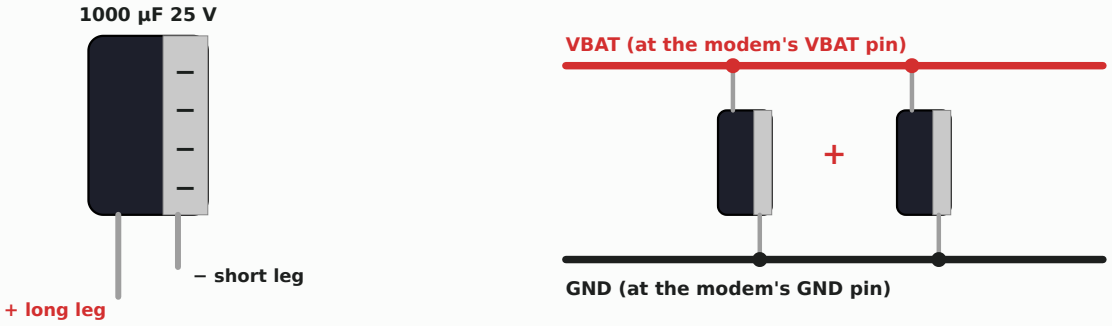


Figure 10. Capacitor polarity. The side with the stripe is **−**. Two capacitors side by side, both + to VBAT and both − to GND.

Steps

1. Find these pins on your board and **write down where they are**: **VBAT** (or VCC 3.4 – 4.2 V / BAT), **GND**, **TXD**, **RXD**, **PWRKEY** (or PWR / PEN / K). Some boards also have **VCCIO** or **3V3**.

2. ⛔ Only a **5V** input and no VBAT pin? **Stop. Send me a photo of both sides first.**

3. Check the shield label says **A7670SA-FASE** (FASE = with GPS).

4. Fuse out. Power wires (keep them **short and straight**): - **red 18 AWG**: power board **tap 5 = hole T16** → modem **VBAT**; - **black 18 AWG**: power board **tap G5 = hole R03** → modem **GND**.

5. Solder the **two 1000 µF capacitors** right at the modem's VBAT and GND: **long leg (+) to VBAT**, **striped side (−) to GND**. Cover the legs with heat-shrink.

6. Signal wires (22 AWG black, about 15 cm, with tape flags): - modem **TXD** → ESP32 board **RX2** pin (GPIO16), flag "TX"; - modem **RXD** → ESP32 board **TX2** pin (GPIO17), flag "RX"; - modem **PWRKEY** → ESP32 board **D4** pin, flag "PWR"; - one thin black wire: modem **GND** → **GND rail**; - if the board has **VCCIO / 3V3**: one wire to the **3V3 rail**.

7. Antennas: LTE antenna → **MAIN** (or LTE); GPS antenna → **GNSS** (or GPS). Press **straight down** until each snaps.

8. Insert the **nano-SIM** (cut corner as drawn on the slot).

9. Fuse in, USB in. Upload **Test 2** below.

Test 2: modem terminal

```
// Ginhawa test 2: talk to the A7670SA. Type AT commands in the Serial Monitor.
// Serial Monitor: 115200 baud, "Both NL & CR".
#define MODEM_RX 16   // ESP32 RX2 <- modem TXD
#define MODEM_TX 17   // ESP32 TX2 -> modem RXD
#define PWRKEY    4   // ESP32 D4  -> modem PWRKEY

void setup() {
  Serial.begin(115200);
  Serial2.begin(115200, SERIAL_8N1, MODEM_RX, MODEM_TX);
  pinMode(PWRKEY, OUTPUT);
  Serial.println("Pressing the modem's power key for 1 second...");
  digitalWrite(PWRKEY, HIGH);
  delay(1000);
  digitalWrite(PWRKEY, LOW);
  Serial.println("Wait about 15 seconds, then type: AT");
}

void loop() {
  while (Serial.available())  Serial2.write(Serial.read());   // you -> modem
  while (Serial2.available()) Serial.write(Serial2.read());   // modem -> you
}
```

10. Serial Monitor: **115200 baud** and **"Both NL & CR"**. Wait **15 seconds**, then type each command and press Enter:

Type	Good answer	Means
`AT`	`OK`	modem is on
`AT+CPIN?`	`+CPIN: READY`	SIM detected
`AT+CSQ`	`+CSQ: 18,99`	signal; first number 10 – 31 is usable
`AT+CEREG?`	`+CEREG: 0,1` or `0,5`	registered on 4G
`AT+CGNSSPWR=1`	`+CGNSSPWR: READY!`	GPS on
`AT+CGPSINFO`	numbers with N and E	position (take it outside, wait 1 – 5 min)

If it fails

What you see	Fix
No answer to `AT`	Swap TXD/RXD wires; measure VBAT at the modem (3.6 – 4.2 V); press the board's own power button if it has one.
`+CPIN: ERROR`	SIM upside down, not registered, or has a PIN (remove it with a phone).
`+CSQ: 99,99`	Antenna on the wrong socket, or no signal indoors: go near a window.
Modem restarts when connecting	Battery low or VBAT wires too long/thin; add the third capacitor.

Stage H – Start the Ginhawa app on the laptop

Time: 30 minutes.

1. Make sure **Docker Desktop** is running (whale icon, "Engine running").

2. Open **PowerShell**: `cd C:\Users\peter\ginhawa`

3. Type `ipconfig`. Under **Wireless LAN adapter Wi-Fi**, copy the **IPv4 Address** (e.g. 192.168.1.20).

4. `notepad .env` and set (with your address):

```
SITE_ADDRESS=localhost, 192.168.1.20
DEFAULT_SNI=192.168.1.20
```

Save. Also copy the value after `ENROLL_SECRET=` (you need it next).

5. `docker compose up -d --build` and wait until it finishes.

> ✅ **Check:** open **https://localhost** in the browser, accept the warning (Advanced → Proceed): the Ginhawa app opens.

Stage I – Upload the Ginhawa program

Time: 30 minutes.

1. In File Explorer open `C:\Users\peter\ginhawa\firmware\ginhawa_node`.

2. Copy `config.example.h`, paste, rename the copy to **`config.h`** (turn on View → File name extensions so it isn't `config.h.txt`).

3. Double-click `ginhawa_node.ino`. Click the **config.h** tab and change:

```
#define WIFI_SSID      "YourWiFiName"
#define WIFI_PASSWORD  "YourWiFiPassword"
#define API_BASE_URL   "https://192.168.1.20"
#define ENROLL_SECRET  "same-as-ENROLL_SECRET-in-.env"
#define LTE_APN        "internet.dito.ph"
```

- Wi-Fi must be **2.4 GHz** (the ESP32 can't see 5 GHz).
- API_BASE_URL = **https://** + your laptop's address, **no slash at the end**.
- APN: DITO `internet.dito.ph`, Smart `internet`, Globe `internet.globe.com.ph`.

4. Save (Ctrl+S). Select **ESP32 Dev Module** + COM port. **Upload**.

5. Serial Monitor at **115200**.

> ✅ **Check: within 2 minutes you see** 1. `Chip ID … will enroll on first upload` 2. `Connecting to YourWiFiName…... connected` 3. `Ginhawa station ready.` 4. `Enrolled as "New station (…)"` 5. `Uploaded 1 reading(s) over Wi-Fi`
>
> In the app, a new station card says **"Just added"**.

If it fails

Message	Fix
`Connecting … failed`	Wrong name/password, or a 5 GHz network.
`Enrollment failed (HTTP 401/403)`	ENROLL_SECRET doesn't match .env exactly.
`Upload failed (HTTP -1)`	Wrong laptop IP, Docker not running, or different Wi-Fi.
`config.h: No such file`	File is named config.h.txt: rename it.

Stage J – 24-hour table test

1. Unplug USB. The station now runs only on its battery.

2. Put the panel in a sunny window or outside; electronics on the table.

3. Check the app every few hours: a reading every minute, PM values change every 5 minutes, battery shown.

4. Breathe near the sensor or light a match nearby (then blow it out): VOC and PM rise, then fall.

5. Touch the converters, modem and fuse holder: **warm is fine, too hot to hold is not.**

6. After 24 hours write down: battery voltage, any gaps in the data. Keep this for the paper.

Stage K – Prepare the box

You still need to buy: an **IP65 ABS junction box**, about **150 × 100 × 70 mm** or bigger.

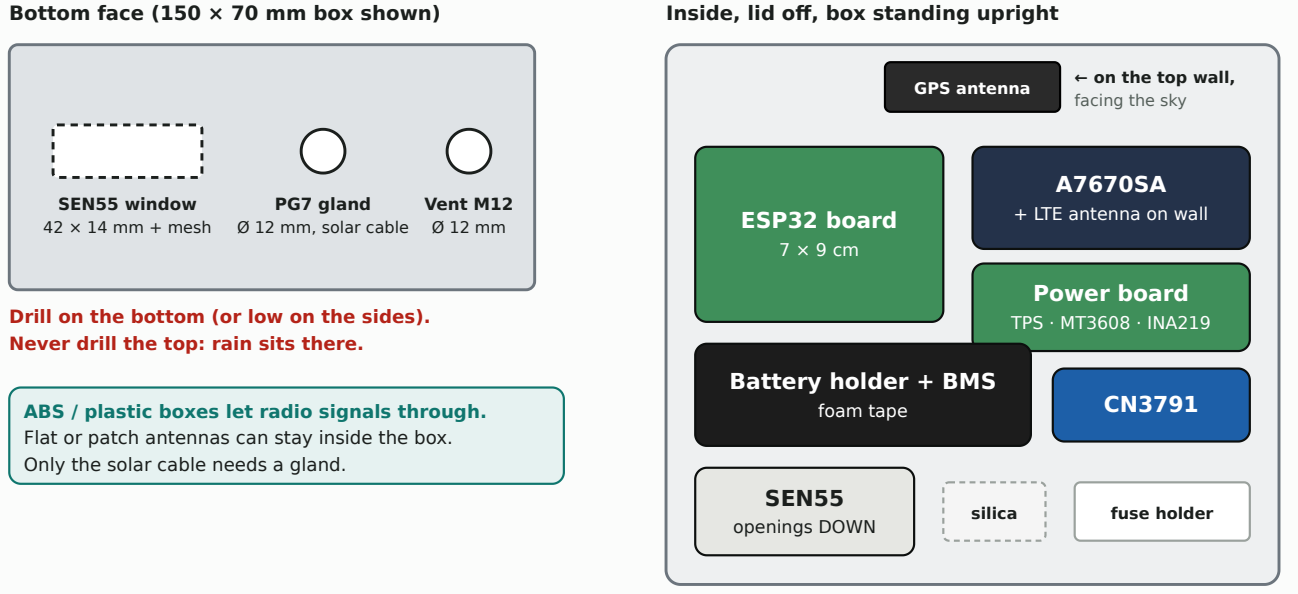


Figure 11. Drilling plan and layout inside an IP65 box of about 150 × 100 × 70 mm. Adjust the positions to the box you buy, keeping these rules.

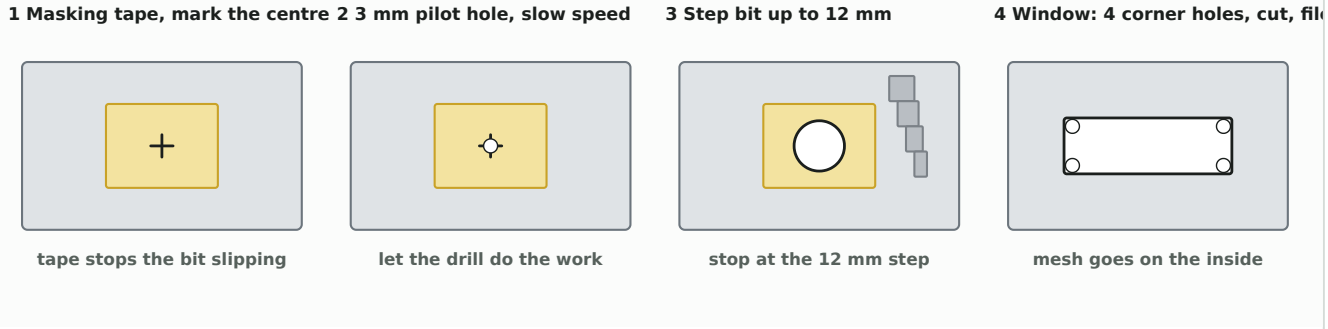


Figure 12. Drilling the plastic box. Go slowly: plastic cracks if you push hard or drill fast.

1. Lay every part inside with the lid on: it must close.

2. On the **bottom face**, put masking tape and mark: **12 mm hole** for the gland, **12 mm hole** for the vent, the **SEN55 window** (42 × 14 mm). Keep 15 mm between holes and edges.

3. Drill **3 mm pilot holes**, slowly, box held firmly.

4. Enlarge to **12 mm** (step bit or 12 mm bit, slow speed).

5. Window: 3 mm holes in the four corners, then cut between them with a cutter (many light cuts). File smooth.

6. Glue the **stainless mesh** on the **inside** of the window, 10 mm bigger on every side, sealed all around.

7. Fit the **PG7 gland** (nut inside) and the **M12 vent** (O-ring outside).

8. Hold the ESP32 board against the back wall, mark its 4 corner holes, drill **3 mm**.

Stage L – Put everything in the box

1. Fuse out, cells out.

2. Screw the **4 standoffs** to the back wall (M3 × 8 screws from outside, 2 mm Allen key). Fit the ESP32 board on them, hold with **M3 nuts**.

3. Foam-tape the **battery holder, power board, CN3791 and modem** to the walls. Zip-tie or tape the holder so cells can't fall out.

4. SEN55: openings facing **down**, against the mesh window. At least **3 cm** from the modem and converters.

5. GPS antenna on the inside of the **top** wall, flat side up. **LTE antenna** on a side wall.

6. Solar cable in through the gland from outside, with a **U-shaped drip loop** outside. Tighten the gland.

7. Zip-tie loose wires. **No bare metal touching anything.**

8. Cells in, fuse in. Connect USB briefly: Serial Monitor shows `Uploaded … over Wi-Fi`. Unplug USB.

9. Add 1 – 2 **silica gel** packs. Close the lid evenly (tighten screws a little at a time, in a cross pattern).

> ✅ **Check:** shake gently: nothing rattles. A new reading appears in the app after closing.

Stage M – Mount it outside and check the app

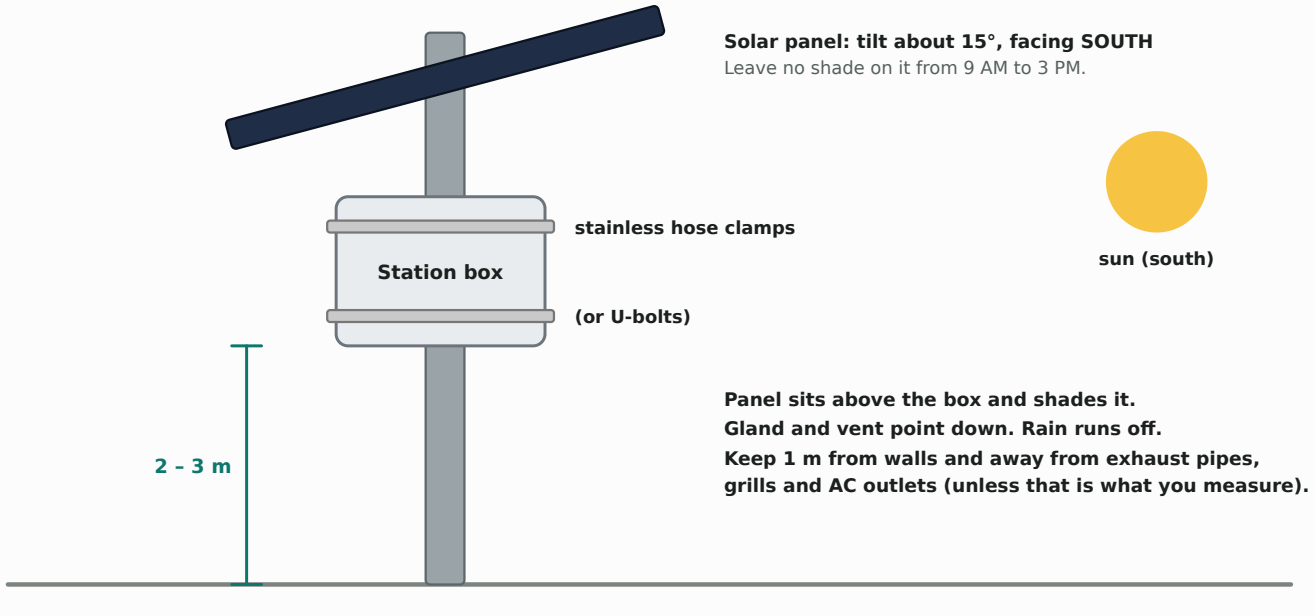


Figure 13. Mounting on a pole. The sensor should breathe the same air people do: 2 to 3 m above the ground, in open air.

1. Choose the spot: open air, **2 – 3 m high**, at least **1 m from walls**, away from exhaust, grills and air-con outlets, with Wi-Fi or mobile signal.

2. Fix the box to the pole with **2 stainless hose clamps** (or U-bolts).

3. Panel **above** the box, tilted about **15°**, facing **south**.

4. Gland and vent **pointing down**. Solar cable zip-tied to the pole with a drip loop.

5. Open the app: readings every minute. Within **15 minutes** outdoors, the GPS finds the position and the station **renames itself** after its barangay.

6. Write down the place, date, chip ID and photos for the paper.

Done. For upkeep (cleaning the panel, replacing silica gel, battery checks) see "Keep it running" in the full manual.